S.G. Ganesh

Silly Programming Mistakes => Serious Harm!

As programmers, we know that almost any software that we use (or write!) has bugs. What we might not be aware of is that many disasters occur because of silly mistakes.

What can software bugs cost? "Nothing," I hear someone saying. They can be beneficial and ensure 'job security'— since the more bugs we put in the software, the more work we get in the future to fix those embedded bugs!

On a more serious note, software bugs can even cost human lives. Many mishaps and disasters have happened in the past because of software bugs; see [1] for a detailed list. For example, during the 1980s, at least six people were killed because of a synchronisation bug in the Therac-25 radiation treatment machine. In 1996, the Ariane 5 rocket exploded shortly after its take-off because of an unhandled overflow exception.

A sobering thought about software bugs is that, though they might occur because of silly or innocuous mistakes, they can cause serious harm.

In 1962, the Mariner-I rocket (meant to explore Venus) veered off track and had to be destroyed. It had a few software bugs and one main problem was traced to the following Fortran statement:

DO 5 K = 1. 3. The "." should have been a comma. The statement was meant to be a DO loop, as in "DO 5 K = 1, 3", but while typing the program, it was mistyped as "DO 5 K = 1. 3".

So, what's the big deal? In old Fortran, spaces were ignored, so we can have spaces in identifiers (yes, believe me, it's true). Hence this became a declaration for a variable of the real type DO5K with an initial value of 1.3 instead of a DO loop. So, a rocket worth $18.5 million was lost because of a typo error!

In 1990, the AT&T long distance telephone network crashed for nine hours because of a software bug. It cost the company millions of dollars. The mistake was the result of a misplaced break statement. The code that was put inside a switch statement looked like the following (from [2]):

```
network code()
{
switch (line) {
case THING1:
        doit1();
        break;
case THING2:
        if (x == STUFF) {
          do_first_stuff();
          if (y == OTHER_STUFF)
            break;
          do_later_stuff();
        } /* coder meant to break to here... */
        initialize_modes_pointer();
        break;
    default:
        processing();
    } /* ...but actually broke to here! */
    unix_modes_pointer();/* leaving the modes_pointer uninitialized */
}
```

As you can see, the programmer has put a "break;" after the *if* condition. He actually wanted to break it outside the *if* condition; but the control gets transferred to outside the (enclosing) switch statement! We all know that it is not possible to use "break" to come outside of an *if* block; this simple mistake resulted in a huge loss to AT&T.

Programmers are usually surprised at how silly mistakes such as the use of the wrong operator symbols, the wrong termination condition for a loop, etc, can lead to serious software problems. True, while most such mistakes will not cause any harm, some minor errors could sometimes lead to major disasters. END

References

- Collection of Software Bugs: www5.in.tum.de/~huckle/bugse.html
- Expert C Programming, Peter van der Linden, Prentice Hall PTR, 1994

About the author:

S.G. Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.